

The Vikings

VS

Saxons

CLASS INHERITANCE
MINI PROJECT





Participants

Keberth José Rodríguez
Wilmer Acosta
Chantal Silva
Eloisa Salinas





BRIEF

The Vikings and Saxons are at war.

Our task was to implement their battle logic in Python using classes, inheritance, and object-oriented programming principles.

Apply Python skills in a comprehensive, real-world scenario, enhancing problem-solving abilities and coding proficiency.

Types of Classes



PARENT CLASS

SOLDIER

CHILD CLASS

VIKING

SAXON

CLASS

WAR





Parent Class

SOLDIER

CONSTRUCTOR

HEALTH

STRENGTH

RECEIVE DAMAGE

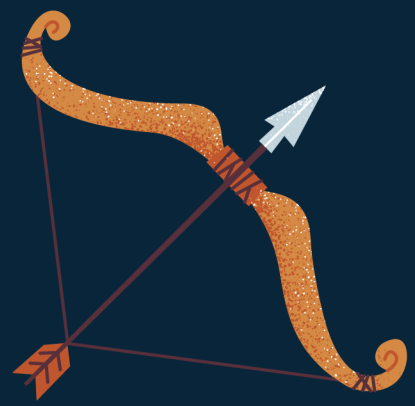
METHODS

```
class Soldier:
    #constructor function
    def __init__(self, health, strength):
        self.health= health
        self.strength= strength

    #attack method
    def attack(self):
        return self.strength

    #receiveDamage method
    def receiveDamage(self, damage):
        self.health -=damage
```

CHILD CLASS

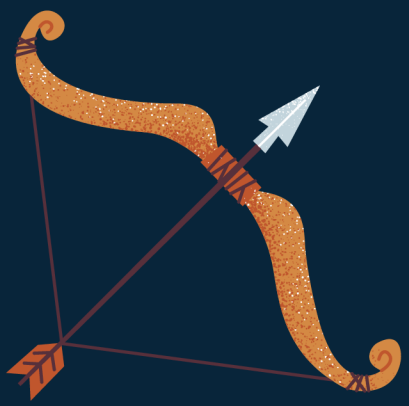


VIKING

There subclasses inherits the same characteristics as Solder (Health and Strength) and both have an attack method and a Damage Estimation, both calculated in the same way. We will use Vikings to illustrate the code for both as it is the same

```
class Viking(Soldier):  
    def __init__(self, name, health, strength):  
        super().__init__(health, strength)  
        self.name = name  
  
    def receiveDamage(self, damage):  
        self.health -= damage  
  
        if self.health <= 0:  
            return f"{self.name} has died in act of combat"  
        else:  
            return f"{self.name} has received {damage} points of damage"  
  
    def battleCry(self):  
        return "Odin Owns You All!"
```

CHILD CLASS



SAXON

If and else conditional statements help us to present 2 possible scenarios:

- a) the health condition of the viking or saxon deteriorates due to the damage suffered, displaying a message
- b) Health goes to the value 0, that means the soldier is dead

```
class Saxon(Soldier):  
    """ ...  
  
    def __init__(self, health, strength):  
        super().__init__(health, strength)  
  
    def receiveDamage(self, damage):  
        self.health -= damage  
        if self.health <= 0:  
            return f"A Saxon has died in combat"  
        else:  
            return f"A Saxon has received {damage} points of damage"
```

Class War



WAR

ARMY CREATION

```
class War():  
    def __init__(self):  
        self.vikingArmy = []  
        self.saxonArmy = []  
        #Method 1  
        def addViking(self, Viking):  
            self.vikingArmy.append(Viking)  
        #Method 2  
        def addSaxon(self, Saxon):  
            self.saxonArmy.append(Saxon)
```

Class War



WAR

Rules of battlefield

- Viking attack
- Saxon attack
- Show status

Viking attack

Saxon attack

Show status

```
Method 3
def vikingAttack(self):
    saxon = random.choice(self.saxonArmy)
    viking = random.choice(self.vikingArmy)
    saxon_receive_damage = saxon.receiveDamage(viking.strength)
    if saxon.health <= 0:
        self.saxonArmy.remove(saxon)
    return saxon_receive_damage
```

```
Method 4
def saxonAttack(self):
    saxon = random.choice(self.saxonArmy)
    viking = random.choice(self.vikingArmy)
    viking_receive_damage = viking.receiveDamage(saxon.strength)
    if viking.health <= 0:
        self.vikingArmy.remove(viking)
    return viking_receive_damage
```

```
Method 5
def showStatus(self):
    if len(self.vikingArmy) == 0:
        return "Saxons have fought for their lives and survive another day..."
    elif len(self.saxonArmy) == 0:
        return "Vikings have won the war of the century!"
    else:
        return "Vikings and Saxons are still in the thick of battle."
```

Thank You!

